PROV-O

溯源本体 · 规范速查

PROV-O — THE PROV ONTOLOGY, ENTITIES & ACTIVITIES

01 定位 · PROV 家族里的本体

WHY PROV-O

「这份数据从哪来、谁在什么时候加工过它」——溯源(provenance)问题。W3C 的 PROV 家族给了一整套答案:概念模型定词汇,本体做 RDF 化,记法定序列化,约束定推理。

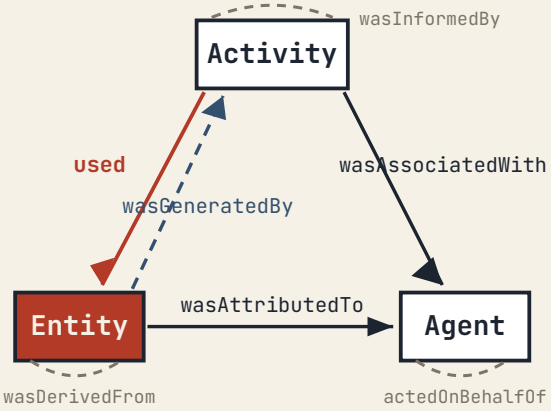
文档	角色	形态
PROV-DM	概念数据模型:实体/活动/代理与影响关系	术语字典(abstract)
PROV-O	把 DM 映射成 OWL 2 本体,RDF 三元组可直接落库	本文速查对象
PROV-N	函数式文本记法:`wasGeneratedBy(e2,a1)`	序列化语法
PROV-CONSTRAINTS	约束与推理规则:等价、有效性判定	形式化规则
PROV-AQ / PROV-LINKS	溯源的访问查询 / 跨文档链接	外围配套

KEY POINT

PROV-O(The PROV Ontology),**W3C Recommendation,2013-04-30**。所有词汇收在命名空间 `prov: = http://www.w3.org/ns/prov#` 下,本体以 **OWL 2 DL** 表达——既是 schema 也是可直接断言的实例词汇。

02 核心三角 · Entity - Activity - Agent

STARTING POINT



实体被活动**使用**与**产出**,代理为活动**担责**,实体向代理**归属**——一条链读完:报告 wasGeneratedBy 编译 · 编译 used 草稿 · 编译 wasAssociatedWith 机器人 · 报告 wasAttributedTo 实验室。

虚线自环是三类「同类关系」:实体⇒实体、活动⇒活动、代理⇒代理。属性名读起来就是主语到宾语的陈述句,方向照着读即可。

起点属性	domain → range	一句话
`prov:used`	Activity → Entity	使用之始(Usage 是"开始用")
`prov:wasGeneratedBy`	Entity → Activity	产出之成(Generation 是"生成完成")
`prov:wasDerivedFrom`	Entity → Entity	新实体由旧实体变换/更新而来
`prov:wasAttributedTo`	Entity → Agent	把实体归属给代理
`prov:wasAssociatedWith`	Activity → Agent	活动指派责任给代理
`prov:actedOnBehalfOf`	Agent → Agent	代理代表另一代理行事(委托)
`prov:wasInformedBy`	Activity → Activity	活动间经由交换实体间接通信
`prov:startedAtTime / endedAtTime`	Activity → xsd:dateTime	数据属性,活动的起/止时刻

03 实体演化 · 一个东西的多种身份

ENTITY EVOLUTION

同一「东西」随时间演进,每个阶段是不同的 Entity——报告的 v1 与 v2 是两个实体,而「贝多芬第五交响曲」与「它的乐谱抄本」是同一事物的两种表述。五个关系负责把它们串起来。

属性	domain → range	何时用
`prov:specializationOf`	Entity → Entity	更具体的特化:同一事物的某个视图/版本
`prov:alternateOf`	Entity → Entity	同一事物的两种表述,可互相替代
`prov:wasRevisionOf`	Entity → Entity	修订关系:改了内容,仍是"它"的下一版
`prov:wasQuotedFrom`	Entity → Entity	引用:这段话摘自那份原文
`prov:hadPrimarySource`	Entity → Entity	多跳派生追到头的原始来源
`prov:wasInvalidatedBy`	Entity → Activity	实体因活动失效(不再"有效",非删除)
`prov:generatedAtTime / invalidatedAtTime`	Entity → xsd:dateTime	实体诞生与失效的时刻(数据属性)

规范细节

属性有子属性链:`specializationOf ⊑ alternateOf`。特化必然是替身,替身未必是特化——"报告 v2"特化自"这份报告",而中文版报告与英文版报告互为替身、谁也不比谁特化。

04 扩展类 · 给三大类补充常见身份

EXPANDED CLASSES

起点三类是骨架,扩展类给骨架穿上日常的衣服——人、组织、软件、集合、位置,全部仍是 Entity / Agent 的子类,不引入新模型。

类	是什么	典型用法
`prov:Collection`	实体的集合	`hadMember` 逐个挂成员(Collection → Entity)
`prov:EmptyCollection`	空集合	Collection 的子类,显式表达"无成员"
`prov:Bundle`	装溯源断言的实体	溯源包本身也是实体,可整体引用、再被溯源
`prov:Person / Organization`	Agent 的两种具象	自然人 / 机构
`prov:SoftwareAgent`	软件形态的代理	流水线每一步的"执行者"都该是它
`prov:Location`	位置	`atLocation` 挂到活动/实体/代理上

工程判断

自动化系统里别把脚本写成 Activity 的属性——建一个 `SoftwareAgent`,用 `wasAssociatedWith` 挂给活动。将来追责、重跑、审计,都有实体可指。

05 其余动词 · 启停、影响与表示

MORE PREDICATES

属性	domain → range	一句话
`prov:wasStartedBy`	Activity → Entity	被某实体触发启动
`prov:wasEndedBy`	Activity → Entity	被某实体触发终止
`prov:influenced / wasInfluencedBy`	任意 → 任意	最泛的影响关系,能用具体的就别用它
`prov:generated / invalidated`	Activity → Entity	上一页两个属性的正式反向(读起来更顺的场合)
`prov:atLocation`	Activity / Entity / Agent / 瞬时事件 → Location	发生在哪
`prov:value`	Entity → 字面量	实体的"直接表示"(如某时刻的读数)

反向三对

规范只正式声明三对 `owl:inverseOf`:`influenced ⇄ wasInfluencedBy`、`generated ⇄ wasGeneratedBy`、`invalidated ⇄ wasInvalidatedBy`。`used` 没有正式反向——查询时自己写 property path。

06 一条完整的溯源链 · Turtle

WORKED EXAMPLE

发布场景:草稿 v1 经 ETL 机器人编译成报告 v2。每个角色各就各位,时间齐备,关联关系带细节节点。

```
@prefix prov: <http://www.w3.org/ns/prov#> .
@prefix xsd:  <http://www.w3.org/2001/XMLSchema#> .
@prefix :     <http://example.com/audit#> .

:report-v2 a prov:Entity ;                  # 产出物
    prov:wasGeneratedBy :compile ;         # 实体 ← 活动
    prov:wasDerivedFrom :draft-v1 ;         # 实体 ← 实体
    prov:wasAttributedTo  :lab ;            # 实体 ← 代理
    prov:generatedAtTime "2026-10-14T09:30:00+08:00"^^xsd:dateTime .

:compile a prov:Activity ;                 # 加工活动
    prov:used :draft-v1 ;                    # 活动 → 实体
    prov:wasAssociatedWith :etl-bot ;       # 活动 → 代理
    prov:startedAtTime "2026-10-14T09:00:00+08:00"^^xsd:dateTime ;
    prov:endedAtTime   "2026-10-14T09:30:00+08:00"^^xsd:dateTime ;
    prov:qualifiedAssociation [             # 给关系一个"细节节点"
        a prov:Association ;
        prov:agent   :etl-bot ;
        prov:hadRole :transformer ] .       # 角色是资源,不是字符串

:etl-bot a prov:SoftwareAgent .            # 软件也是代理
:lab     a prov:Organization .
```

```
// 同一条链的 PROV-N 写法 —— PROV-DM 的函数式记法,起止时间走位置参数
document
  prefix : <http://example.com/audit#>
  entity(:draft-v1)  entity(:report-v2)
  activity(:compile, 2026-10-14T09:00:00+08:00, 2026-10-14T09:30:00+08:00)
  used(:compile, :draft-v1, -)
  wasGeneratedBy(:report-v2, :compile, -)
  wasDerivedFrom(:report-v2, :draft-v1)
  wasAttributedTo(:report-v2, :lab)
  wasAssociatedWith(:compile, :etl-bot, -)
  agent(:etl-bot, [prov:type="prov:SoftwareAgent"])  agent(:lab, [prov:type="prov:Organization"])
endDocument
```

PROV-O

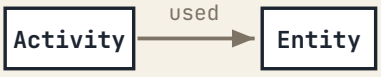
溯源本体 · 规范速查

PROV-O — QUALIFICATION, INFLUENCE & PITFALLS

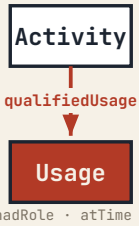
07 限定修饰模式 · 给二元关系加细节

QUALIFICATION

RDF 的边只有两个端点,"何时用、以何角色、凭何计划"挂不上去。限定修饰的解法:给这层关系一个独立节点——二元说「用了」,限定说「在 t 时刻、以转换角色、按某计划用」。



二元 · 一句话



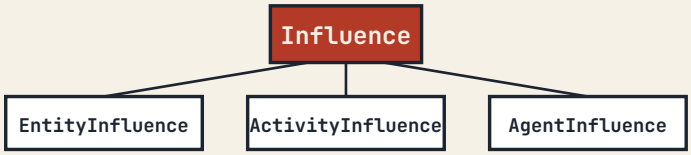
hadRole · atTime

升级规则:qualifiedX 都是对应二元属性的**子属性**,range 是固定的限定类;细节(hadRole、atTime、hadPlan…)全部挂在限定类节点上。

二元属性	限定属性	限定类	引用属性
used	qualifiedUsage	Usage	entity
wasGeneratedBy	qualifiedGeneration	Generation	activity
wasDerivedFrom	qualifiedDerivation	Derivation	entity
wasStartedBy	qualifiedStart	Start	entity
wasEndedBy	qualifiedEnd	End	entity
wasInvalidatedBy	qualifiedInvalidation	Invalidation	activity
wasInformedBy	qualifiedCommunication	Communication	activity
wasAssociatedWith	qualifiedAssociation	Association	agent + hadPlan/hadRole
wasAttributedTo	qualifiedAttribution	Attribution	agent
actedOnBehalfOf	qualifiedDelegation	Delegation	agent + hadActivity
hadPrimarySource	qualifiedPrimarySource	PrimarySource	entity
wasQuotedFrom	qualifiedQuotation	Quotation	entity
wasRevisionOf	qualifiedRevision	Revision	entity
wasInfluencedBy	qualifiedInfluence	Influence	influencer

08 影响的统一理论 · 一切皆 Influence

INFLUENCE HIERARCHY



entity

Usage · Derivation
Start · End
PrimarySource
Quotation · Revision

activity

Generation · Communication
Invalidation

agent

Attribution · Association
Delegation

Plan · Role 挂在 Association 上 · 裸 Influence 用 influencer

限定类按「影响源是什么」分三支,每支用固定的**引用属性**指回影响源:entity、activity、agent;拿不准影响源类型时用裸 Influence + influencer。

规范建议

`wasInfluencedBy` 太泛,规范明说:能用更具体的子属性就用子属性;同理,别只断言裸 `Influence` 类型——信息量太低,查询方还要猜。

09 实操坑清单

PITFALLS

01 qualified 与二元**双写**:qualifiedX 虽是二元的子属性,但只写限定,不识该模式的工具就漏读;两写都留最稳。

02 prov:wasUsedBy 不存在:used 没有正式反向,查询用 property path 或退到 wasInfluencedBy。

03 Role / Plan 是**资源**不是字符串:角色建 IRI(:transformer),别塞字面量。

04 时间一律 xsd:dateTime 且带时区——不然跨系统排序没有意义。

05 开放世界:查不到 wasDerivedFrom 不等于没派生——溯源查询是找证据,不是证没有。

06 别人的溯源包整体当 Bundle 引用,别拆散混进自己的图——断言的出处就断了。

07 失效(wasInvalidatedBy)不是删除:实体仍在图里,只是过了有效期。

08 属性方向别背反:主语是"结果方"才用 was* 系列;used 的主语永远是活动。